空间数据质量检查工具 - 技术文档

一、项目概述

1.1 项目名称

空间数据质量检查工具 (Spatial Data Quality Check Tool)

1.2 项目目标

设计开发用于空间数据质量检查的应用软件工具，该工具运行于单PC机桌面环境，不需要网络连接，支持地图数据加载、可视化显示、错误标记记录和错误查询等功能。

1.3 技术架构

采用分层架构模式，分为表示层(UI)、业务逻辑层(Core)和数据访问层(Data)三层结构。

二、开发环境配置

2.1 系统要求

项目	要求
操作系统	Windows 10/11
Python版本	Python 3.9+
内存	建议4GB以上
硬盘	至少100MB可用空间

2.2 依赖库安装

```
# 创建虚拟环境（可选）
python -m venv venv
.\venv\Scripts\activate

# 安装依赖
pip install -r requirements.txt
```

2.3 requirements.txt 内容

```
PyQt5>=5.15.0
matplotlib>=3.5.0
numpy>=1.21.0
Pillow>=9.0.0
PyYAML>=6.0
```

2.4 项目目录结构

```
SpatialDataQC/
│
├── main.py                     # 程序入口
├── config.yaml                 # 配置文件
├── requirements.txt            # 依赖清单
├── README.md                   # 帮助文档
│
├── ui/                         # 表示层
│   ├── __init__.py
│   ├── main_window.py          # 主窗口
│   ├── map_canvas.py           # 地图画布组件
│   ├── error_panel.py          # 错误列表面板
│   ├── text_viewer.py          # 文本查看器
│   ├── query_dialog.py         # 查询对话框
│   └── error_dialog.py         # 错误编辑对话框
│
├── core/                       # 业务逻辑层
│   ├── __init__.py
│   ├── data_loader.py          # 数据加载服务
│   ├── error_manager.py        # 错误管理服务
│   └── query_engine.py         # 查询引擎
│
├── data/                       # 数据访问层
│   ├── __init__.py
│   ├── file_reader.py          # 文件读取器
│   ├── error_store.py          # 错误存储器
│   └── models/                 # 数据模型
│       ├── __init__.py
│       ├── error_record.py     # 错误记录模型
│       └── map_data.py         # 地图数据模型
│
├── utils/                      # 工具模块
│   ├── __init__.py
│   ├── logger.py               # 日志工具
│   ├── file_utils.py           # 文件工具
│   └── coordinate.py           # 坐标转换工具
│
├── resources/                  # 资源文件
│   ├── icons/                  # 图标
│   └── styles/                 # 样式表
│
├── test_data/                  # 测试数据
│   ├── sample_data.csv
│   ├── sample_text.txt
│   ├── sample_map.png
│   └── countries.geojson
│
└── logs/                       # 日志目录
```

三、依赖库详解

3.1 PyQt5

用途：GUI框架，用于构建桌面应用程序界面

主要模块：

模块	用途
QtWidgets	提供UI组件（窗口、按钮、列表等）
QtCore	核心功能（信号槽、事件处理）
QtGui	图形绘制（QPainter、QPixmap、QColor等）

项目中使用的主要类：

- `QMainWindow`：主窗口类
- `QWidget`：基础控件类
- `QPainter`：绘制工具类
- `QPixmap`：图像类
- `QImage`：图像数据类
- `pyqtSignal`：信号机制

3.2 NumPy

用途：数值计算，处理栅格数据

主要功能：

- 多维数组操作
- 数据类型转换
- 数值计算

项目中的应用：

```
import numpy as np

# 栅格数据存储
raster_data: np.ndarray

# 数据类型转换
data = ((data - data.min()) / (data.max() - data.min()) * 255).astype(np.uint8)
```

3.3 Pillow (PIL)

用途：图像处理，加载栅格数据文件

主要类:

- `Image`: 图像对象

项目中的应用:

```
from PIL import Image

# 加载图像文件
img = Image.open(file_path)

# 转换为NumPy数组
raster_array = np.array(img)

# 保存图像
Image.fromarray(img).save(path)
```

3.4 PyYAML

用途: 解析YAML配置文件

项目中的应用:

```
import yaml

# 读取配置
with open('config.yaml', 'r', encoding='utf-8') as f:
    config = yaml.safe_load(f)
```

3.5 Python标准库

库	用途
`json`	JSON文件读写，错误记录存储
`os`	文件路径操作
`datetime`	时间处理
`dataclasses`	数据类定义
`enum`	枚举类型定义
`typing`	类型注解
`logging`	日志记录

四、数据模型层 (data/models/)

4.1 error_record.py - 错误记录模型

ErrorType 枚举类

说明：定义错误类型

```
class ErrorType(Enum):
    GEOMETRY = "几何错误"       # 图形形状问题
    ATTRIBUTE = "属性错误"     # 属性值问题
    TOPOLOGY = "拓扑错误"      # 空间关系问题
    COORDINATE = "坐标错误"    # 坐标值问题
    OTHER = "其他"             # 其他类型
```

方法：

方法	参数	返回值	说明
`from_code(code: str)`	错误类型代码	ErrorType	根据代码字符串获取枚举值

ErrorStatus 枚举类

说明：定义错误处理状态

```
class ErrorStatus(Enum):
    PENDING = "未处理"
    PROCESSING = "处理中"
    RESOLVED = "已解决"
    IGNORED = "已忽略"
```

方法：

方法	参数	返回值	说明
`from_str(status: str)`	状态字符串	ErrorStatus	根据中文名称获取枚举值

ErrorRecord 数据类

说明：错误记录数据模型，存储单条错误信息

属性：

属性名	类型	说明
`error_id`	str	错误编号（自动生成，格式：ERR-YYYYMMDD-XXXX）
`error_type`	ErrorType	错误类型
`description`	str	错误描述
`position`	Tuple[float, float]	错误位置坐标(x, y)

属性名	类型	说明
`layer_name`	str	所在图层名称
`created_time`	datetime	创建时间
`file_path`	str	关联的地图文件路径
`status`	ErrorStatus	处理状态（默认：未处理）
`remark`	Optional[str]	备注信息
`modified_time`	Optional[datetime]	最后修改时间

方法:

方法	参数	返回值	说明
`to_dict()`	无	Dict[str, Any]	转换为字典格式
`from_dict(data)`	字典数据	ErrorRecord	从字典创建实例（类方法）
`to_json()`	无	str	转换为JSON字符串

4.2 map_data.py - 地图数据模型

DataType 枚举类

说明：定义数据类型

```
class DataType(Enum):
    RASTER = "栅格数据"
    VECTOR = "矢量数据"
    TEXT = "文本数据"
    UNKNOWN = "未知类型"
```

MapData 数据类

说明：地图数据模型，存储加载的地图数据

属性:

属性名	类型	说明
`file_path`	str	文件路径
`data_type`	DataType	数据类型
`file_name`	str	文件名
`width`	int	宽度（像素或字符）

属性名	类型	说明
`height`	int	高度（像素或行数）
`extent`	Optional[Tuple[float, float, float, float]]	数据范围(x_min, y_min, x_max, y_max)
`crs`	Optional[str]	坐标参考系统
`raster_data`	Optional[np.ndarray]	栅格数据数组
`vector_features`	Optional[List[dict]]	矢量要素列表
`text_content`	Optional[str]	文本内容
`text_lines`	Optional[List[str]]	文本行列表

属性方法:

属性	返回值	说明
`is_loaded`	bool	判断数据是否已加载

方法:

方法	参数	返回值	说明
`get_pixel_at(x, y)`	坐标	Optional[Tuple]	获取指定位置的像素值
`get_line_at(line_number)`	行号	Optional[str]	获取指定行的文本
`to_display_info()`	无	dict	获取用于显示的信息字典

五、数据访问层 (data/)

5.1 file_reader.py - 文件读取器

FileReader 类

说明: 负责读取各种格式的地图数据文件

类属性:

属性	类型	说明
`SUPPORTED_RASTER`	set	支持的栅格格式扩展名
`SUPPORTED_VECTOR`	set	支持的矢量格式扩展名
`SUPPORTED_TEXT`	set	支持的文本格式扩展名

类方法:

方法	参数	返回值	说明
`get_data_type(file_path)`	文件路径	DataType	判断文件数据类型

方法	参数	返回值	说明
`is_supported(file_path)`	文件路径	bool	判断文件是否支持
`load_file(file_path)`	文件路径	Optional[MapData]	加载文件并返回MapData对象
`_load_raster(map_data)`	MapData	MapData	加载栅格数据（私有方法）
`_load_vector(map_data)`	MapData	MapData	加载矢量数据（私有方法）
`_load_text(map_data)`	MapData	MapData	加载文本数据（私有方法）

加载流程:

```
load_file()
    ├── 判断文件类型
    ├── 创建MapData对象
    ├── 根据类型调用加载方法
    │   ├── _load_raster() → 使用PIL.Image加载
    │   ├── _load_vector() → 解析GeoJSON
    │   └── _load_text() → 读取文本内容
    └── 返回MapData对象
```

5.2 error_store.py - 错误存储器

ErrorStore 类

说明: 负责错误记录的存储和管理

属性:

属性	类型	说明
`storage_dir`	str	存储目录
`_errors`	Dict[str, ErrorRecord]	错误记录字典
`_current_file`	Optional[str]	当前文件路径

方法:

方法	参数	返回值	说明
`set_storage_dir(directory)`	目录路径	None	设置存储目录
`get_error_file_path(map_file_path)`	地图文件路径	str	获取对应的错误文件路径
`save_errors(file_path)`	文件路径（可选）	bool	保存错误记录到JSON文件

方法	参数	返回值	说明
`load_errors(file_path)`	文件路径	List[ErrorRecord]	从JSON文件加载错误记录
`add_error(error)`	ErrorRecord	None	添加错误记录
`update_error(error_id, **kwargs)`	错误ID, 更新字段	bool	更新错误记录
`delete_error(error_id)`	错误ID	bool	删除错误记录
`get_error(error_id)`	错误ID	Optional[ErrorRecord]	获取指定错误
`get_all_errors()`	无	List[ErrorRecord]	获取所有错误
`get_errors_by_type(error_type)`	错误类型	List[ErrorRecord]	按类型筛选错误
`get_errors_by_status(status)`	状态	List[ErrorRecord]	按状态筛选错误
`search_errors(keyword)`	关键字	List[ErrorRecord]	搜索错误
`clear_errors()`	无	None	清除所有错误
`get_error_count()`	无	int	获取错误数量
`get_statistics()`	无	Dict[str, int]	获取统计信息

六、业务逻辑层 (core/)

6.1 data_loader.py - 数据加载服务

DataLoaderService 类

说明：封装数据加载业务逻辑

属性：

属性	类型	说明
`_current_data`	Optional[MapData]	当前加载的数据
`_file_reader`	FileReader	文件读取器实例

方法：

方法	参数	返回值	说明
`load_map_data(file_path)`	文件路径	Optional[MapData]	加载地图数据
`get_current_data()`	无	Optional[MapData]	获取当前数据
`is_data_loaded()`	无	bool	判断是否已加载数据
`get_data_info()`	无	Optional[dict]	获取数据信息

方法	参数	返回值	说明
`clear_current_data()`	无	None	清除当前数据
`get_supported_formats()`	无	dict	获取支持的格式列表

6.2 error_manager.py - 错误管理服务

ErrorManagerService 类

说明：封装错误管理业务逻辑

属性：

属性	类型	说明
`_error_store`	ErrorStore	错误存储器实例
`_counter`	int	错误编号计数器
`_current_map_file`	Optional[str]	当前地图文件路径

方法：

方法	参数	返回值	说明
`set_current_map_file(file_path)`	文件路径	None	设置当前地图文件
`create_error(error_type, description, position, layer_name)`	错误信息	ErrorRecord	创建新错误记录（自动编号）
`update_error(error_id, **kwargs)`	错误ID, 更新字段	bool	更新错误记录
`delete_error(error_id)`	错误ID	bool	删除错误记录
`get_error(error_id)`	错误ID	Optional[ErrorRecord]	获取指定错误
`get_all_errors()`	无	List[ErrorRecord]	获取所有错误
`get_errors_by_type(error_type)`	错误类型	List[ErrorRecord]	按类型获取错误
`get_errors_by_status(status)`	状态	List[ErrorRecord]	按状态获取错误
`save_errors(file_path)`	文件路径（可选）	bool	保存错误记录

方法	参数	返回值	说明
`load_errors(file_path)`	文件路径	List[ErrorRecord]	加载错误记录
`get_error_count()`	无	int	获取错误数量
`get_statistics()`	无	dict	获取统计信息
`clear_errors()`	无	None	清除所有错误
`get_error_store()`	无	ErrorStore	获取错误存储器实例

错误编号规则:

```
格式: ERR-YYYYMMDD-XXXX
示例: ERR-20260326-0001
      ERR-20260326-0002
```

6.3 query_engine.py - 查询引擎

QueryService 类

说明: 封装错误查询业务逻辑

属性:

属性	类型	说明
`_error_manager`	ErrorManagerService	错误管理服务实例

方法:

方法	参数	返回值	说明
`search_by_keyword(keyword)`	关键字	List[ErrorRecord]	按关键字搜索
`search_by_id(error_id)`	错误ID	Optional[ErrorRecord]	按ID精确查询
`filter_by_type(error_type)`	错误类型	List[ErrorRecord]	按类型筛选
`filter_by_status(status)`	状态	List[ErrorRecord]	按状态筛选
`filter_by_position_range(x_min, y_min, x_max, y_max)`	范围坐标	List[ErrorRecord]	按位置范围筛选

方法	参数	返回值	说明
`advanced_search(keyword, error_type, status)`	多个条件	List[ErrorRecord]	高级组合查询
`get_sorted_errors(by_field, ascending)`	排序字段, 升序	List[ErrorRecord]	获取排序后的错误列表

七、表示层 (ui/)

7.1 main_window.py - 主窗口

MainWindow 类

继承：QMainWindow

说明：应用程序主窗口，整合所有功能模块

信号：

信号	参数	说明
`data_loaded`	str	数据加载完成信号
`error_created`	str	错误创建完成信号

属性：

属性	类型	说明
`data_loader`	DataLoaderService	数据加载服务
`error_manager`	ErrorManagerService	错误管理服务
`query_service`	QueryService	查询服务
`map_canvas`	MapCanvas	地图画布
`text_viewer`	TextViewer	文本查看器
`error_panel`	ErrorPanel	错误面板
`is_marking_mode`	bool	是否处于标记模式

主要方法：

方法	说明
`_init_ui()`	初始化UI界面
`_create_menu_bar()`	创建菜单栏
`_create_tool_bar()`	创建工具栏
`_create_central_widget()`	创建中心部件

方法	说明
`_create_status_bar()`	创建状态栏
`_connect_signals()`	连接信号槽
`open_file()`	打开文件对话框
`load_file(file_path)`	加载指定文件
`open_error_file()`	打开错误记录文件
`save_errors()`	保存错误记录
`toggle_marking_mode()`	切换标记模式
`on_map_clicked(x, y)`	地图点击事件处理
`on_error_selected(error_id)`	错误选中事件处理
`delete_error(error_id)`	删除错误
`edit_error(error_id)`	编辑错误
`show_query_dialog()`	显示查询对话框
`show_statistics()`	显示统计信息
`show_help_document()`	打开帮助文档
`show_about()`	显示关于对话框

7.2 map_canvas.py - 地图画布

MapCanvas 类

继承：QWidget

说明：地图可视化显示组件，支持栅格和矢量数据显示

信号：

信号	参数	说明
`position_changed`	float, float	鼠标位置变化信号
`point_clicked`	float, float	地图点击信号
`error_marker_clicked`	str	错误标记点击信号

属性：

属性	类型	说明
`map_data`	Optional[MapData]	地图数据
`pixmap`	Optional[QPixmap]	显示的图像

属性	类型	说明
`error_markers`	Dict[str, ErrorRecord]	错误标记字典
`zoom_factor`	float	缩放因子
`offset_x`	float	X轴偏移
`offset_y`	float	Y轴偏移
`is_marking_mode`	bool	是否标记模式

主要方法:

方法	参数	返回值	说明
`load_data(map_data)`	MapData	None	加载地图数据
`_load_raster_data(map_data)`	MapData	None	加载栅格数据
`_load_vector_data(map_data)`	MapData	None	加载矢量数据
`set_marking_mode(enabled)`	bool	None	设置标记模式
`add_error_marker(error)`	ErrorRecord	None	添加错误标记
`remove_error_marker(error_id)`	str	None	移除错误标记
`clear_error_markers()`	无	None	清除所有标记
`highlight_error(error)`	ErrorRecord	None	高亮显示错误
`zoom_in()`	无	None	放大
`zoom_out()`	无	None	缩小
`fit_to_window()`	无	None	适应窗口
`screen_to_map(x, y)`	屏幕坐标	Tuple[float, float]	屏幕坐标转地图坐标
`map_to_screen(x, y)`	地图坐标	Tuple[int, int]	地图坐标转屏幕坐标

事件处理:

事件	说明
`paintEvent`	绑制事件，绑制地图和错误标记
`mousePressEvent`	鼠标按下事件
`mouseMoveEvent`	鼠标移动事件
`mouseReleaseEvent`	鼠标释放事件
`wheelEvent`	滚轮事件，用于缩放
`resizeEvent`	窗口大小变化事件

7.3 error_panel.py - 错误面板

ErrorPanel 类

继承：QWidget

说明：错误记录列表面板，显示和管理错误记录

信号：

信号	参数	说明
`error_selected`	str	错误选中信号
`error_delete_requested`	str	请求删除错误信号
`error_edit_requested`	str	请求编辑错误信号

属性：

属性	类型	说明
`_errors`	List[ErrorRecord]	错误列表
`error_list`	QListWidget	列表控件
`type_filter`	QComboBox	类型筛选下拉框
`status_filter`	QComboBox	状态筛选下拉框

主要方法：

方法	参数	说明
`update_errors(errors)`	更新错误列表	
`add_error(error)`	添加单个错误	
`remove_error(error_id)`	移除错误	
`refresh_errors(errors)`	刷新错误列表	
`clear_errors()`	清除所有错误	
`select_error(error_id)`	选中指定错误	

7.4 text_viewer.py - 文本查看器

TextViewer 类

继承：QWidget

说明：文本数据显示组件，用于显示TXT/CSV文件

信号：

信号	参数	说明
`line_clicked`	int	行点击信号

属性:

属性	类型	说明
`map_data`	Optional[MapData]	地图数据
`lines`	List[str]	文本行列表
`highlighted_line`	int	高亮行号
`text_edit`	QTextEdit	文本编辑控件

主要方法:

方法	参数	说明
`load_text(map_data)`	加载文本数据	
`highlight_line(line_number)`	高亮指定行	
`clear_highlight()`	清除高亮	

7.5 query_dialog.py - 查询对话框

QueryDialog 类

继承: QDialog

说明: 错误查询对话框

信号:

信号	参数	说明
`error_selected`	str	错误选中信号

主要方法:

方法	说明
`search()`	执行搜索
`_display_results()`	显示搜索结果
`_goto_selected()`	定位到选中的错误

7.6 error_dialog.py - 错误编辑对话框

ErrorDialog 类

继承：QDialog

说明：错误信息录入和编辑对话框

主要方法：

方法	返回值	说明
`get_error_info()`	Tuple[ErrorType, str]	获取输入的错误信息

八、工具模块 (utils/)

8.1 logger.py - 日志工具

Logger 类

说明：单例模式的日志记录器

方法：

方法	说明
`debug(message)`	记录调试信息
`info(message)`	记录一般信息
`warning(message)`	记录警告信息
`error(message)`	记录错误信息
`critical(message)`	记录严重错误

日志文件位置：`logs/qc_YYYYMMDD.log`

8.2 file_utils.py - 文件工具

函数列表：

函数	参数	返回值	说明
`ensure_dir(directory)`	目录路径	str	确保目录存在
`get_file_extension(file_path)`	文件路径	str	获取文件扩展名
`get_file_name(file_path)`	文件路径	str	获取文件名
`get_file_dir(file_path)`	文件路径	str	获取文件目录
`is_file_exists(file_path)`	文件路径	bool	判断文件是否存在

函数	参数	返回值	说明
`get_unique_filename(directory, base_name, extension)`	目录, 基名, 扩展名	str	获取唯一文件名
`format_file_size(size_bytes)`	字节数	str	格式化文件大小

8.3 coordinate.py - 坐标转换工具

函数列表:

函数	参数	返回值	说明
`screen_to_world(screen_x, screen_y, offset_x, offset_y, scale)`	屏幕坐标, 偏移, 缩放	Tuple[float, float]	屏幕坐标转世界坐标
`world_to_screen(world_x, world_y, offset_x, offset_y, scale)`	世界坐标, 偏移, 缩放	Tuple[float, float]	世界坐标转屏幕坐标
`normalize_coordinates(x, y, extent)`	坐标, 范围	Tuple[float, float]	归一化坐标
`denormalize_coordinates(norm_x, norm_y, extent)`	归一化坐标, 范围	Tuple[float, float]	反归一化坐标
`is_point_in_extent(x, y, extent)`	坐标, 范围	bool	判断点是否在范围内
`calculate_distance(x1, y1, x2, y2)`	两点坐标	float	计算两点距离

九、数据流与交互流程

9.1 数据加载流程

```
用户操作: 文件 → 打开地图数据
    │
    ▼
MainWindow.open_file()
    │
    ▼
QFileDialog.getOpenFileName()  ← 用户选择文件
    │
    ▼
MainWindow.load_file(file_path)
    │
    ▼
DataLoaderService.load_map_data(file_path)
    │
    ▼
FileReader.load_file(file_path)
```

```
    │
    ├── _load_raster()  → 栅格数据
    ├── _load_vector()  → 矢量数据
    └── _load_text()    → 文本数据
    │
    ▼
返回 MapData 对象
    │
    ▼
MapCanvas.load_data(map_data)  或  TextViewer.load_text(map_data)
    │
    ▼
显示数据
```

9.2 错误标记流程

```
用户操作: 点击"标记错误"按钮
    │
    ▼
MainWindow.toggle_marking_mode()
    │
    ▼
MapCanvas.set_marking_mode(True)
    │
    ▼
用户点击地图位置
    │
    ▼
MapCanvas.point_clicked.emit(x, y)
    │
    ▼
MainWindow.on_map_clicked(x, y)
    │
    ▼
ErrorDialog.exec_()  ← 用户填写错误信息
    │
    ▼
ErrorManagerService.create_error(...)
    │
    ├── 生成错误编号 (ERR-YYYYMMDD-XXXX)
    ├── 创建 ErrorRecord 对象
    └── 存储到 ErrorStore
    │
    ▼
MapCanvas.add_error_marker(error)
    │
    ▼
ErrorPanel.add_error(error)
    │
    ▼
更新界面显示
```

9.3 错误查询流程

```
用户操作: 工具 → 查询错误
    │
    ▼
MainWindow.show_query_dialog()
    │
    ▼
QueryDialog.exec_()
    │
    ▼
用户输入关键字/选择筛选条件
    │
    ▼
QueryDialog.search()
    │
    ▼
QueryService.advanced_search(...)
    │
    ▼
返回匹配的 ErrorRecord 列表
    │
    ▼
QueryDialog._display_results()
    │
    ▼
用户双击结果
    │
    ▼
QueryDialog.error_selected.emit(error_id)
    │
    ▼
MainWindow.on_error_selected(error_id)
    │
    ▼
MapCanvas.highlight_error(error)
```

十、配置文件说明

config.yaml

```yaml
app:
  name: "空间数据质量检查工具"
  version: "1.0.0"
  author: "开发者"

display:
  window_width: 1200
```

```yaml
    window_height: 800
    default_zoom: 1.0

  error_types:
    - code: "GEOMETRY"
      name: "几何错误"
      color: "#FF0000"
    - code: "ATTRIBUTE"
      name: "属性错误"
      color: "#00FF00"
    # ...

  supported_formats:
    raster: [".tif", ".tiff", ".bmp", ".png", ".jpg", ".jpeg"]
    vector: [".shp", ".geojson"]
    text: [".txt", ".csv"]
```

十一、错误记录文件格式

JSON文件结构

```json
{
  "version": "1.0",
  "saved_time": "2026-03-26 10:30:00",
  "errors": [
    {
      "error_id": "ERR-20260326-0001",
      "error_type": "GEOMETRY",
      "error_type_name": "几何错误",
      "description": "多边形边界不闭合",
      "position": [116.4074, 39.9042],
      "layer_name": "默认图层",
      "created_time": "2026-03-26 10:25:30",
      "file_path": "D:/data/map.tif",
      "status": "未处理",
      "remark": null
    }
  ]
}
```

十二、扩展开发指南

12.1 添加新的数据格式支持

1. 在 `FileReader` 类中添加格式扩展名：

```
SUPPORTED_RASTER = {'.tif', '.tiff', '.new_format'}
```

2. 实现加载方法：

```
@classmethod
def _load_new_format(cls, map_data: MapData) -> MapData:
    # 实现新格式加载逻辑
    return map_data
```

3. 在 `load_file` 方法中添加分支：

```
elif data_type == DataType.NEW_FORMAT:
    return cls._load_new_format(map_data)
```

12.2 添加新的错误类型

1. 在 `ErrorType` 枚举中添加：

```
class ErrorType(Enum):
    # 现有类型...
    NEW_TYPE = "新类型错误"
```

2. 在 `config.yaml` 中添加配置：

```
error_types:
  - code: "NEW_TYPE"
    name: "新类型错误"
    color: "#FF00FF"
```

3. 在 `MapCanvas._draw_error_markers` 中添加颜色映射：

```
elif error.error_type.name == "NEW_TYPE":
    color = QColor(255, 0, 255)
```

十三、版本信息

- **版本号**：v1.0.0
- **发布日期**：2026年3月26日
- **开发语言**：Python 3.9+
- **GUI框架**：PyQt5 5.15+

文档最后更新：2026年3月26日